



عملي

3

15

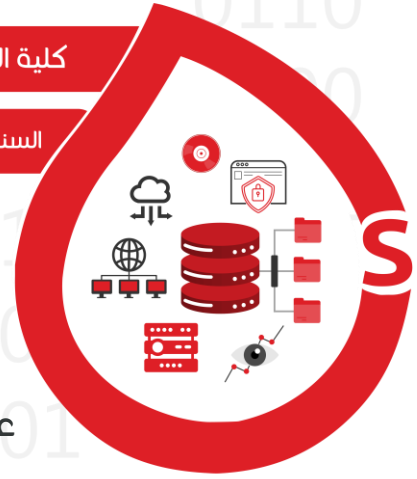
2250 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Clustered Index

عملي مشترك



RB Informatcs; 10/11/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

Clustered index

تحدثنا في المحاضرة السابقة عن عملية البحث والفرق بين ال seek وال scan .
ولكن هل سيتغير لدي البحث وكلفته اذا تم الامر باستخدام الفهارس؟
هذا ما سنستكشفه معاً

■ كيف يتم انشاء ال clustered index ؟

عن طريق استخدام التعليمة الاتية :

```
CREATE CLUSTERED INDEX name
ON
table_name (column);
```

■ ماذا تعني هذه التعليمة؟؟

■ معنى هذه التعليمة هو بناء فهرس على الجدول حسب الاعمدة التي يمكننا تحديدها (يمكننا تحديد اكثر من عامود) لتسهيل عملية البحث التي نقوم بها

وهذا النوع من ال index ليس له كيان خاص به أي لا يقوم بتشكيل object خاص به وانما نقوم بتركيبه على الجدول نفسه اذا ان ال optimizer يقوم بإعادة ترتيب السجلات بناء على الاعمدة التي قمنا بتحديددها وسنكمل في محاضرتنا الحديث عنه

لَيْسَ الْعِلْمُ مَا حُفِظَ، إِنَّمَا الْعِلْمُ مَا نَفَع.

مثال عملي :

سنقوم بإنشاء الجدول الآتي والذي يحتوي على :

```
CREATE TABLE EMP1 (
  id INT IDENTITY(1,1) PRIMARY KEY,
  First_name VARCHAR(255),
  Last_name VARCHAR(255),
  Gender VARCHAR(10),
  Salary INT
);
```

قمنا بإنشاء الجدول حيث يحتوي جدول على :

1. Id رقم تعريف الموظف وهو رقم فريد لكل موظف
2. First_name وهو الاسم الأول للموظف
3. Last_name الاسم الأخير للموظف
4. Gender
5. Salary راتب الموظف

شرح مبسط لكل عمود:

1. id: (العمود الأساسي (Primary Key) ، يستخدم للتعريف الفريد لكل موظف، ويُرتب غالباً البيانات عند استخدام الـ Clustered Index.
2. First_name و Last_name: لتخزين أسماء الموظفين.
3. Gender: لتحديد الجنس، مهم إذا أردنا إجراء استعلامات أو ترتيب البيانات حسب الجنس.
4. Salary: يمكننا استخدامه للترتيب أو لإجراء عمليات حسابية مثل المتوسط أو الحد الأقصى.
5. IDENTITY(1,1) تعني أن العمود id سيبدأ من 1 ويزيد تلقائياً بواحد لكل سجل جديد.
6. PRIMARY KEY يجعل العمود فريد ويستخدم غالباً لبناء الـ clustered index بشكل افتراضي.
7. Gender اخترت VARCHAR(10) لأن غالباً سيكون 'M' أو 'F'.

مثال:

```
INSERT INTO EMP1 (First_name, Last_name, Gender, Salary) VALUES
('Ali', 'Ahmad', 'M', 3000),
('Sara', 'Khan', 'F', 4000),
('Omar', 'Ahmad', 'M', 3500),
('Lina', 'Saleh', 'F', 4500);
```

والآن سنقوم بمجموعة استعلامات :

1. استعلام لجلب كل السجلات بعد إنشاء Clustered Index على id

```
CREATE CLUSTERED INDEX idx_emp_id ON EMP1(id);
```

وتعني:

إنشئ فهرس Clustered باسم idx_emp_id على جدول EMP1 بحيث يتم ترتيب كل الصفوف حسب العمود id لتسريع البحث والترتيب

- idx_emp_id ال
- هذا اسم الفهرس الذي أنشأته (يمكننا تسميته أي اسم مناسب)

ولاننسى بان ال CLUSTERED INDEX

نوع من الفهرس (Index) الذي يرتب البيانات نفسها داخل الجدول حسب العمود المحدد

الاستعلام لجلب كل البيانات:

```
SELECT * FROM EMP1;
```

SELECT * FROM EMP1;

129 %

Results Messages

	id	First_name	Last_name	Gender	Salary
1	1	Ali	Ahmad	M	3000
2	2	Sara	Khan	F	4000
3	3	Omar	Ahmad	M	3500
4	4	Lina	Saleh	F	4500



Clustered Index يعني أن البيانات في الجدول سيتم ترتيبها فعلياً حسب العمود id.

عند عمل `SELECT *` ستري البيانات مرتبة تلقائياً حسب id بسبب الـ clustered index.

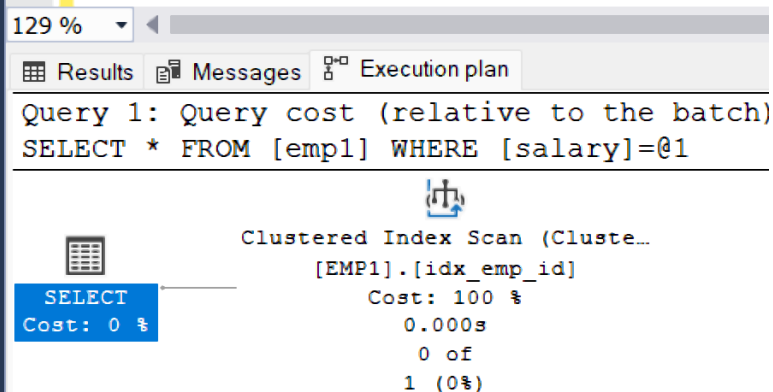
ملاحظة: ((حتى لو أدخلت البيانات بشكل عشوائي وغير مرتب حسب id، بعد إنشاء Clustered Index على العمود id

عند عمل `SELECT *` بدون أي `ORDER BY`، ستري البيانات مرتبة حسب id تلقائياً))

2. لو أعدنا الاستعلام الذي يحتوي على الـ `where` لن نستفيد وذلك لأن الـ clustered مبني على الـ id والاستعلام لدينا على الـ salary

```
Select*
From emp1
Where salary=10000;
```

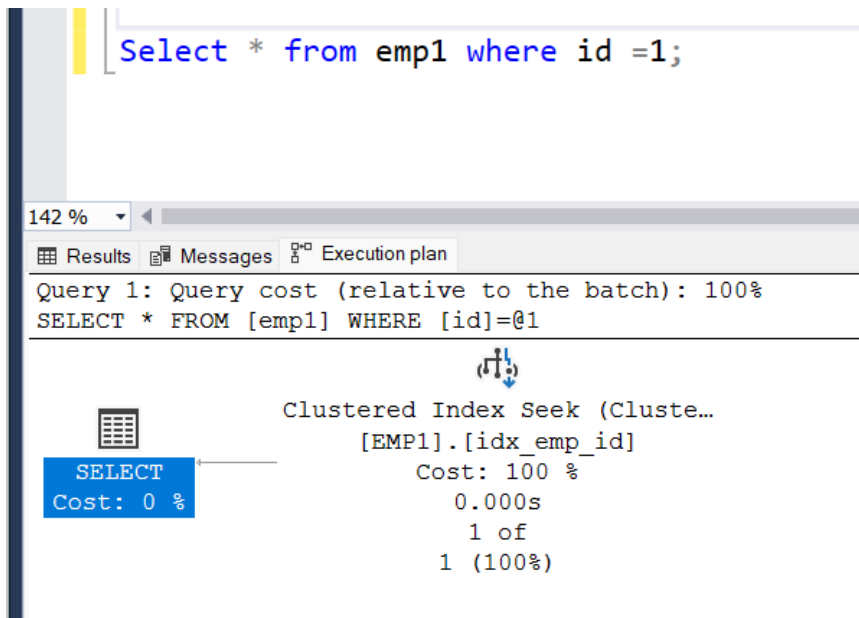
```
Select*
From emp1
Where salary=10000;
```



نلاحظ أن الـ cost يساوي 100%

3. لو قمنا باستعلام يحوي ال where على ال id نلاحظ انه تغير ال clustered index من ال scan ال ال seek

Select * from emp1 where id =1;



لاحظ أن ال select cost =0% وال clustered index seek =100%

لماذا ؟

لان ال scan كان يقوم بمسح الجدول كاملا بينما ال seek يسمح ال records مباشرة
اذا هو موجه أي يمكن تشبيه ال index لمبدأ ال Binary search
هنا حددنا السجل الذي يملك id = 1
مفيد للوصول المباشر لسجل معين

4. استعلام مع شرط على Last_name :

SELECT *
FROM EMP1
WHERE Last_name = 'Ahmad'

Where يقوم بجلب السجلات التي تطابق الاسم الأخير وهو احمد.

```
SELECT *
FROM EMP1
WHERE Last_name = 'Ahmad'
```

142 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

SELECT * FROM [EMP1] WHERE [Last_name]=@1

Clustered Index Scan (Cluste...
[EMP1].[idx_emp_id]
Cost: 100 %
0.000s
2 of
2 (100%)

SELECT
Cost: 0 %



5. استعمال باستخدام BETWEEN :

```
SELECT *
FROM EMP1
WHERE id BETWEEN 3 AND 5;
```

```
SELECT *
FROM EMP1
WHERE id BETWEEN 3 AND 5;
```

42 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

SELECT * FROM [EMP1] WHERE [id]>=@1 AND [id]<=@2

Clustered Index Seek (Cluste...
[EMP1].[idx_emp_id]
Cost: 100 %
0.000s
2 of
2 (100%)

SELECT
Cost: 0 %



clustered index seek cost100% وال select cost=0%



▪ سؤال ؟ هل سيقوم هنا بعمل ال seek ام ال scan؟

نلاحظ بأنه قام بعمل seek

6. إنشاء Clustered Index جديد على Salary و Gender

ملاحظة 1:

يجب حذف ال clustered index القديم (إذا كان موجوداً)
ومن ثم انشاء ال CLUSTERED INDEX الجديد
وذلك لأنه يجب ان يكون لدي واحد فقط

```
DROP INDEX idx_emp_id ON EMP1;
```

ملاحظة 2

الطريقة الصحيحة لإنشاء ال index
يجب تحديد اسم الجدول ثم اسم ال index

1. اسم الجدول:

- ضروري، لأن SQL Server يحتاج أن يعرف على أي جدول يبني الفهرس.
- في المثال EMP1: هو الجدول

2. اسم ال: Index

- مطلوب أيضاً، لأنه كل Index يحتاج اسماً فريداً في قاعدة البيانات.
- في المثال idx_emp_gender_salary: هو اسم الفهرس الجديد.
- يمكنك اختيار أي اسم وصفي، لكن يجب أن يكون مختلف عن أي Index موجود على نفس الجدول

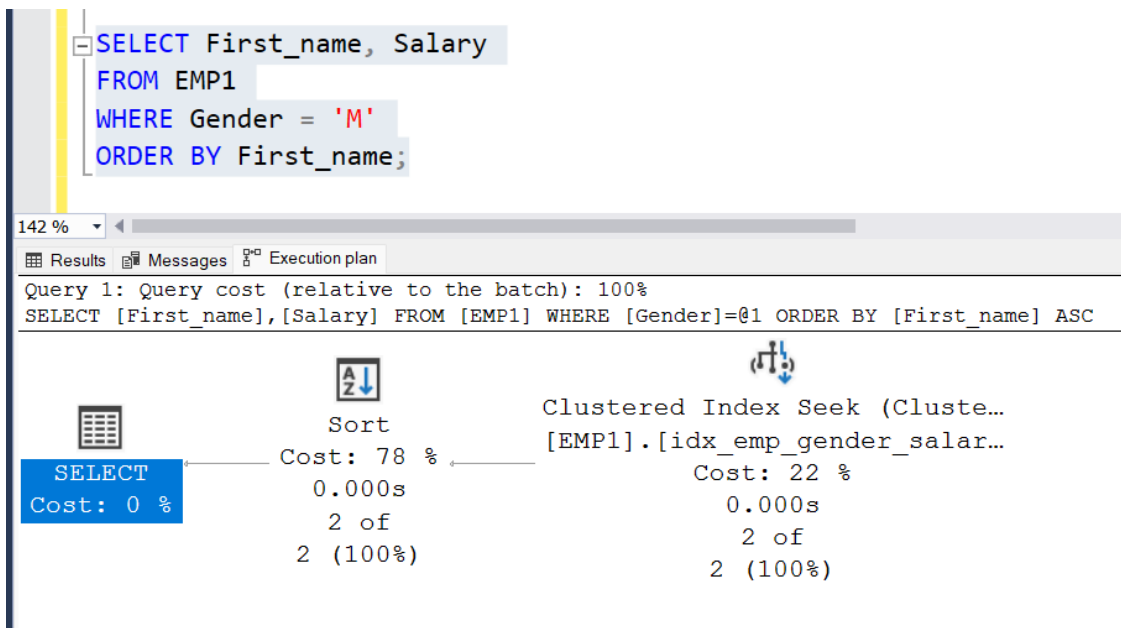
```
CREATE CLUSTERED INDEX idx_emp_gender_salary  
ON EMP1(Gender, Salary);
```

يمكننا هنا اختيار أسلوب الترتيب ان كان تصاعدي او تنازلي

الآن البيانات سيتم ترتيبها أولاً حسب Gender ثم حسب Salary
أي M و F سيتم ترتيبهم أولاً، وبعدها الراتب داخل كل جنس.

select firstname and the salary for men

```
SELECT First_name, Salary
FROM EMP1
WHERE Gender = 'M'
ORDER BY First_name;
```



شرح:

- قمنا بجلب العمودين First_name و Salary فقط.
- الشرط (criteria) هو: Gender = 'M'.
- بما أن الـ Clustered Index مبني على (Gender, Salary)، فإن شرط Gender يطابق أول مفتاح في الفهرس، لذلك يستطيع SQL Server استخدام Clustered Index Seek.
- يقوم الـ Seek بالوصول مباشرة إلى الصفوف التي قيمتها Gender = 'M' دون الحاجة لقراءة الجدول بالكامل.
- بعد جلب الصفوف، يتم تطبيق ORDER BY First_name مما يؤدي إلى إعادة ترتيب النتائج حسب الاسم الأول أبجدياً، حتى وإن كان ترتيب الفهرس مختلفاً.
- الترتيب النهائي لا يعتمد على ترتيب الفهرس بل على ORDER BY.

7. تغيير ال ORDER BY إلى Salary

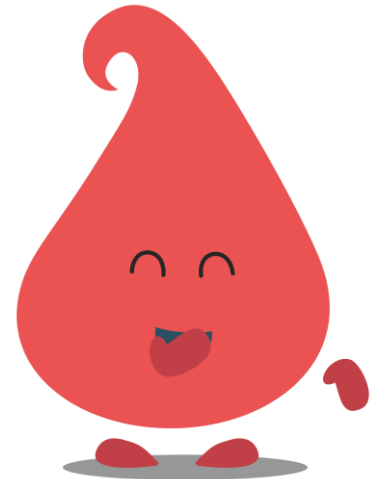
```
SELECT First_name, Salary
FROM EMP1
WHERE Gender = 'M'
ORDER BY Salary;
```

Query 1: Query cost (relative to the batch): 100%

SELECT [First_name],[Salary] FROM [EMP1] WHERE [Gender]=('M') ORDER BY [Salary]

Clustered Index Seek (Cluste...
[EMP1].[idx_emp_gender_salar...]
Cost: 100 %
0.000s
2 of
2 (100%)

SELECT
Cost: 0 %



■ في حال قمنا بتغيير ال order by إلى salary يختفي ال sort لكون ال clustered مبني على ال gender وال salary

حذف Clustered Index القديم وإعادة بناءه على id

```
DROP INDEX idx_emp_gender_salary ON EMP1;
```

عندما نقوم باستعلام لجلب كافة ال records

Select First_name from dbo.EMP1 where Salary >=1000

And First_name like 'ahmad'

رج تعطيني ال index scan لماذا؟

142 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

Select First_name from dbo.EMP1 where Salary >=1000 And First_name like

Clustered Index Scan (Cluste...
[EMP1].[idx_emp_gender_salar...
Cost: 100 %
0.000s
0 of
1 (0%)

SELECT
Cost: 0 %



لأن الفهرس Clustered Index مبني على Salary و Gender

أما إذا بدنا يكون Index Seek نبني

CREATE CLUSTERED INDEX idx_emp_salary_fname

ON EMP1(Salary, First_name);

سؤال ؟

Select First_name from dbo.EMP1 where salary>=1000

And First_name like '%ahmad';

هل ستعطي ال index scan ولماذا؟

% في بداية النص يعني أي قيمة ممكن تكون قبل ahmad

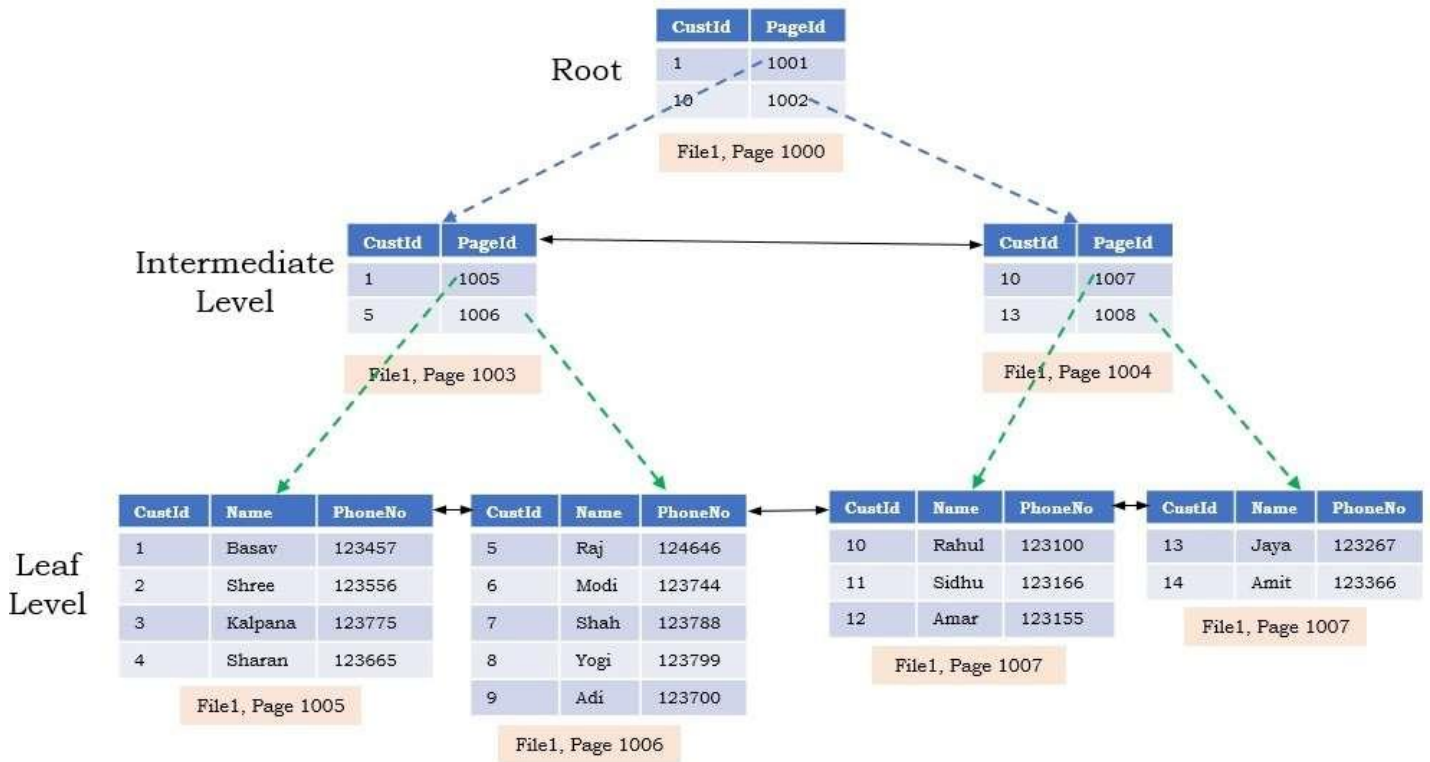
محرك SQL ما يقدر يستخدم ال Index بنفس الكفاءة لأنه لازم يفحص كل الصفوف (scan) عشان يشوف مين الاسماء اللي تنتهي ب ahmad

Execution Plan: Index Scan يقرأ كل الصفوف في الجدول أو في ال (Index)

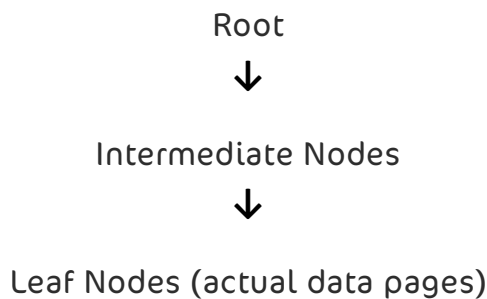
مراجعة

B+ Tree structure of a clustered Index

B+ Tree Structure of a Clustered Index



نلاحظ أنها تتألف من ثلاث مستويات :



- نلاحظ أن الجدول Table موجود في الـ Row leaf level إذ أنه يحمل الـ row كامل فهو يحمل الداتا ذاتها وليس مؤشر عليها
- ولذلك يعد الـ (clustered Index) مركبًا على الجدول .

بالعودة الى المثال السابق ولنوضح كيف تمكن من التوجه المباشر ل ال row بعملية ال seek وذلك باستخدام مخطط ال B+ TREE

بدءا من الجذر root تتم عملية تقسيم الصفحات paging

(كل صفحة تحمل في داخلها أكثر من row)

وكل قسم في ال root يملك مؤشر منه الى level intermediate حيث يتم تقسم الصفحات الى اجزاء اصغر حتى نصل الى ال leaf level الذي نريده

■ وبالعودة الى مخططنا السابق

نريد البحث عن الموظف ذو الرقم 9

1. نبدأ من الجذر اولاً ونلاحظ انه لدينا مؤشرين اثنين

الاول يشير الى الاعداد الاكبر من ال1

والثاني الى الاعداد التي اكبر من ال10

لذلك سوف نذهب الى المؤشر الاول نحو اليسار

2. في ال intermediate level يوجد لدينا مؤشرين:

اول مؤشر يشير الى الاعداد التي اكبر من 1

الثاني يشير الى الاعداد التي اكبر من ال 5 واصغر من ال10

وهنا سوف يذهب الى المؤشر الثاني ونكون قد وصلنا الى الموظف الذي قمنا بالبحث عن (الموظف رقم 9)

اي اصبح لدينا index seek اي مباشرة

اي انه يمكننا القول انه ال Optimizer قام ب scan في ال leaf level ومسح جميع ال records ضمن اخر page ليصل الى الموظف المطلوب

إذا تم اختصار عملية البحث في كامل تسجيلات لل البحث في كل level لذلك ال seek موجه.

كل Leaf Page تحتوي الصفوف مرتبة حسب ال Index Key .

ملاحظات:

- يتم التقسيم الى pages بناء على حجم الداتا التي لدينا وحجم ال row وليس حسب العدد
- اذا قمنا بعمل اضافة (insert) يقوم ال Optimizer باضافته في مكانه الصحيح لتبقى ال data مرتبة ولو كانت ال pages ممتلئة.
- سيقوم بانشاء page جديدة ويضيف لها مؤشر ليحافظ على الترتيب
- نحصل على حالة seek عندما يتوافق ال criteria مع ال index key

- لكي يكون ال Clustered Index أكثر كفاءة يجب أن يكون ال index key فريداً (unique). ليس بالضرورة أن يكون Id ولكن من المهم أن يكون unique لأنه في حال قمنا باختيار name مثلاً سيتم مسح كل تكرارات الاسم في حالة ال Seek ، وفي هذه الحالة على الرغم من أن ال Seek موجه سيكون أقل كفاءة.
- في الحياة العملية، إعادة بناء ال Clustered Index لقاعدة البيانات عملية غير عملية إطلاقاً لأن ذلك يعني إعادة ترتيب ملايين السجلات بناءً على index key جديد، وهو ما يستغرق ساعات.
- لو قمنا ببناء أكثر من Clustered Index على نفس الجدول نحصل على خطأ (error) يمنعنا من عمل Drop لل Clustered Index الحالي قبل إنشاء واحد جديد، وفي حال عمل Drop لل Clustered سيحافظ الجدول على الترتيب.

189 %
Messages

Msg 1902, Level 16, State 3, Line 23
Cannot create more than one clustered index on table 'EMP1'.

- لو قمنا بعمل Insert بعد إزالة ال Clustered Index لن يهتم بالترتيب.
- أسوأ أنواع ال operation ضمن ال execution هي ال sort لأنه في حال كانت ال result set كبيرة فإن إعادة ترتيبه تستهلك ال memory.
- لذلك عند بناء ال index نحاول تجنب عملية ال sort قدر الامكان وكذلك تجنب ال clustered scan

خصائص ال Clustered Index :

- ال Clustered Index يعيد ترتيب الجدول بالكامل عند بنائه، يقوم بترتيب السجلات فعلياً داخل الجدول وفق ال index key.
- يخزن البيانات في ال leaf level من شجرة ال B+Tree أي أن ال leaf node تحتوي على ال Rows نفسها بالكامل.
- يتم تقسيم البيانات إلى صفحات (pages) ويتم الوصول إلى ال Row المطلوب عبر مستويات الشجرة: Root → Intermediate → Leaf.
- يؤدي إلى عمليات Seek سريعة عندما يتوافق الاستعلام مع ال index key فالوصول يكون مباشر إلى الصفحات المتعلقة بالقيم المطلوبة.
- لا يمكن إنشاء أكثر من Clustered Index واحد على نفس الجدول لأن الجدول نفسه يصبح مبنياً ومرتباً بناءً على هذا المؤشر.

6. إعادة بناء الـ Clustered Index عملية مكلفة جداً

لأنها تعني إعادة ترتيب السجلات كلها، وقد تستغرق ساعات إذا كان الجدول كبيراً.

7. عمليات Insert قد تسبب إنشاء صفحات جديدة

وذلك للحفاظ على ترتيب البيانات، وخاصة عند امتلاء الصفحات. (page split)

8. وجود فراغات (gaps) في الصفحات قد يتطلب إعادة ترتيب الشجرة

لذلك يُفضّل تجنب ملء الصفحات بالكامل لتقليل مشاكل إعادة البناء.

9. الـ Clustered Index يعتمد على key يكون من الأفضل أن يكون قريب من الـ unique

لأنه في حال كان غير فريد (مثل اسم الشخص)، سيؤدي ذلك لمسح كل التكرارات عند إجراء Seek ، فيصبح أقل كفاءة مقارنة بعمود مثل Id.

❖ كيف تتخزن البيانات عند إدخالها فيها؟

1. بداية الجدول HEAP :

عندما نقوم بإنشاء جدول جديد بدون Clustered Index ، فإن SQL Server يخزن البيانات فيه على شكل Heap.
 ▪ ما هو الـ Heap ؟

▪ هو جدول بدون ترتيب داخلي.

▪ الصفوف تُخزن حسب الترتيب الذي تُدخل فيه، ولكن هذا الترتيب غير مضمون 100 % لأنه قد يتغير إذا امتلأت الصفحات أو إذا تم نقل صفوف أثناء التحديثات.

▪ الوصول للبيانات يحتاج Table Scan لأن SQL لا يملك أي مفتاح مرتب يساعده على البحث.

▪ ماذا يحدث عند إنشاء الـ clustered index?

عند إنشاء Clustered Index على أي عمود مثال الـ ID يحدث التالي:

1. SQL Server يأخذ كل بيانات الـ Heap

2. يعيد ترتيبها فعلياً على القرص حسب العمود المستخدم في الـ Clustered Index.

3. يبني هيكلية B-Tree ويصبح الجدول Clustered Table بدلاً من Heap

نتيجة هذا الترتيب:

يصبح للجدول ترتيب فعلي للصفوف بحسب مفتاح الفهرس.

أي عملية بحث على العمود المفهرس تستخدم Index Seek بدل من الـ Scan.

أي INSERT جديد يوضع مباشرة في المكان الصحيح داخل الترتيب.

هل يختلف لدينا ال table scan عن ال clustered index scan ؟

1. Table Scan في الHeap

يأتي عندما لا يوجد أي Clustered Index. SQL يقرأ كل الصفحات من البداية إلى النهاية للبحث عن البيانات. يحدث حتى لو طلبت صف واحد فقط.

2. Clustered Index Scan

يحدث عندما يكون هناك Clustered Index لكن الاستعلام لا يستطيع استخدام Seek. SQL يقرأ البيانات من الفهرس المرتب، لكنه يقرأ معظم الصفوف وليس كلها دائماً. هو أفضل قليلاً من Table Scan لأنه يستخدم بنية B-Tree وليس تخزين عشوائي.

العنصر	HEAP (بدون Clustered Index)	Clustered Index (جدول مرتب)
ترتيب الصفوف داخل الصفحات	غير مرتب - حسب توفر المكان	مرتب حسب مفتاح الفهرس
طريقة اختيار الصفحة عند INSERT	أي صفحة فيها مساحة فارغة	الصفحة الصحيحة حسب ترتيب المفتاح
إعادة ترتيب البيانات	لا يعاد ترتيب أي صف	يتم إعادة ترتيب كل البيانات عند إنشاء ال Clustered
الوصول للصفوف	Table Scan فقط	Seek أو Scan حسب الشرط
بنية التخزين	Heap Structure عشوائي	B-Tree Structure منظم
صفحة البيانات (Data Page)	الصفوف مخزنة بشكل عشوائي	الصفوف مخزنة بترتيب المفتاح
Row Offset Table	موجود لترقيم الصفوف داخل الصفحة لكن بدون ترتيب	موجود لكن مفهرس ومنظم حسب المفتاح
التعامل مع الامتلاء	ينتقل إلى صفحة جديدة	قد يقوم ب Page Split للحفاظ على الترتيب
أفضل استخدام لـ	جداول صغيرة أو مؤقتة	جداول كبيرة وكثيرة البحث

متى نقوم باستخدام ال clustered index ؟

- عندما يكون العمود فريد
- عندما يكون رقماً أو تاريخاً
- عندما يكون عموداً يُستخدم بكثرة في WHERE
- عندما يكون مدخلاته متزايدة (ID) مثلاً (لتقليل Page Split)

"هكذا نكون قد تعرفنا على Clustered Index وكيف يتم ترتيب البيانات داخل الجدول لتحسين أداء البحث. في المحاضرة القادمة، سنتعرف على Non-Clustered Index، وهو نوع آخر من الفهارس يساعدنا على تسريع الاستعلامات بشكل ذكي دون تغيير ترتيب البيانات داخل الجدول."